Practice Set: Constructors & Destructors in Inheritance

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Explain why a strict construction contract (top-down lifecycle order) is required when instantiating compound derived class objects in RAM.

Question 2: Outline the core execution lifecycle rules for constructor initialization and destructor teardown when creating and destroying derived class objects.

Question 3: Explain how default constructors are implicitly synthesized and invoked by the C++ compiler in derived classes.

Question 4: What compilation error occurs when a base class defines a parameterized constructor but lacks a default constructor, and the derived class fails to call it explicitly? How does the **Member Initialization List** resolve this?

Question 5: Detail the execution order of constructors and destructors in **Single Inheritance** (class B : public A).

Question 6: Detail the execution order of constructors and destructors in **Multilevel Inheritance** (class C : public B where B : public A).

Question 7: Explain how constructor and destructor execution order is determined in **Multiple Inheritance** (class C : public A, public B). Which structural declaration rule governs the base constructor call sequence?

Question 8: Explain why destructors operate in a bottom-up (LIFO) teardown sequence in class hierarchies.

Question 9: Explain the senior developer best practice regarding **Virtual Destructors** in polymorphic base classes. What resource leak issue arises if a base class destructor is not declared virtual when deleting via a base pointer?

Question 10: Write a complete C++ program demonstrating parameterized constructor forwarding and destructor order in single inheritance. Show base class A accepting an integer parameter and derived class B passing arguments via its Member Initialization List.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): In what directional order (top-down or bottom-up) do constructors execute when an object of a derived class is created?

Question 12 (Viva): In what directional order (top-down or bottom-up) do destructors execute when a derived object goes out of scope?

Question 13 (Viva): If a base class has a default constructor, does the derived class need to explicitly call it in its initialization list?

Question 14 (Viva): What syntax is required in a derived constructor to forward parameters to a base class parameterized constructor?

Question 15 (Viva): In multiple inheritance (class C : public A, public B), does constructor execution depend on the order of base classes in the class header declaration or the member initialization list?

Question 16 (Viva): What is the constructor execution order for a multilevel hierarchy A $\rightarrow$ B $\rightarrow$ C?

Question 17 (Viva): What is the destructor execution order for a multilevel hierarchy A $\rightarrow$ B $\rightarrow$ C?

Question 18 (Viva): What happens if an independent object of a base class is instantiated regarding derived class constructors and destructors?

Question 19 (Viva): Why is the derived destructor body executed before the base class destructor during object destruction?

Question 20 (Viva): What keyword should be added to a base class destructor declaration when deleting derived objects through base class pointers (Base* ptr = new Derived(); delete ptr;)?

Answers & Explanations

Answer: 01

Compound Structure: A derived class object is a composite structure containing both the base class sub-object and the derived class's own members.

Construction Contract: Because derived class methods and attributes often rely upon or interact with inherited base data members, the foundation (base class sub-object) must be fully built and initialized in RAM before the upper derived structure can be assembled.

Answer: 02

Constructor Order (Top-Down): When a derived object is created, the base class constructor executes first, followed immediately by the derived class constructor.

Destructor Order (Bottom-Up / LIFO): When a derived object goes out of scope, the teardown is inverted: the derived class destructor executes first, followed by the base class destructor.

Answer: 03

Implicit Invocation: When a base class has a default constructor (or no parameters), the derived class constructor does not need to explicitly call it.

Compiler Synthesis: The C++ compiler automatically synthesizes and injects the default base constructor call (: A()) into the derived constructor's initialization list prior to executing the derived constructor body.

Answer: 04

Fatal Compilation Error: If a base class only defines parameterized constructors and lacks a default constructor, the compiler cannot guess default arguments, resulting in a fatal compilation error if the derived constructor does not explicitly invoke the base constructor.

Solution via Member Initialization List: The derived class constructor explicitly passes arguments to the base class constructor using member initialization list syntax: DerivedClass(int x, int y) : BaseClass(y) { ... }.

Answer: 05

Constructor Sequence: Base constructor A() executes first, followed by derived constructor B().

Destructor Sequence: Derived destructor ~B() executes first, followed by base destructor ~A().

Answer: 06

Constructor Sequence: Grandparent A() ----> Parent B() ----> Child C().

Destructor Sequence: Child ~C() -----> Parent ~B() -----> Grandparent ~A().

Answer: 07

Constructor Sequence: Evaluated in the exact order the base classes are declared in the **class header**, not their position in the initialization list. For class C : public A, public B, the execution order is A() ----> B() ----> C().

Destructor Sequence: Executes in the exact reverse order of class header declaration: ~C() ----> ~B() ----> ~A().

Answer: 08

Outside-In Teardown: Destructors tear objects down from the outside in.

Resource Cleanup: The derived destructor executes first to release derived-level resources (such as heap allocations or file handles). Once finished, the compiler automatically calls the base destructor to clean up base-level resources, ensuring derived cleanups do not access already-destroyed base sub-objects.

Answer: 09

Polymorphic Destruction Risk: Deleting a derived object through a base pointer (Base* ptr = new Derived(); delete ptr;) when the base destructor is non-virtual results in calling *only* the base destructor.

Resource Leak: The derived class destructor ~Derived() is skipped entirely, leaking any heap memory or resources managed by the derived class.

Best Practice: Always declare the base class destructor as virtual (virtual ~Base() {}) to ensure proper LIFO destructor chaining during polymorphic deletions.

Answer: 10

```
#include <iostream>
using namespace std;
class A
{
  private:
    int a;public:
    // Parameterized constructor for Base Class A
    A(int k)
    {
        a = k;
        cout << "Base Class A initialized with a = " << a << endl;
    }
    ~A()
    {
        cout << "Destructor ~A (Base cleans up second)" << endl;
    }
};
class B : public A
{
  private:
    int b;public:
    // Member Initialization List forwards argument 'y' to Base A
    B(int x, int y) : A(y)
    {
        b = x;
        cout << "Derived Class B initialized with b = " << b << endl;
    }
    ~B()
    {
        cout << "Destructor ~B (Derived cleans up first)" << endl;
    }
};
int main()
{
```

```
    {
        // Passes 7 to Base A(7), assigns 5 to Derived b
        B obj(5, 7);
    } // Out of scope: triggers bottom-up destructor order
    return 0;
}
```

Answer: 11
Top-down order (Base class constructor first, then Derived class constructor).

Answer: 12
Bottom-up / LIFO order (Derived class destructor first, then Base class destructor).

Answer: 13
No; the compiler automatically synthesizes and inserts the default base constructor call into the initialization list.

Answer: 14
The **Member Initialization List** syntax (e.g., Derived(int x) : Base(x)).

Answer: 15
It depends strictly on the order of base classes declared in the **class header**.

Answer: 16
A() -----> B() -----> C().

Answer: 17
~C() -----> ~B() ------> ~A().

Answer: 18
Only the constructor and destructor of that base class are invoked; derived constructors and destructors are not executed.

Answer: 19
To allow derived-level resources to be safely released while base class sub-objects are still fully intact and valid in memory.

Answer: 20
The **virtual** keyword (virtual ~Base() {}).